INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

SBT-DF204 — Computer Forensics Case Studies

Case Study 4: Investigating Rhino Possession Evidence

Student Name	Mary-Joy Adewole
Registration Number	2025/FWSD/11468
Course Code	SBT-DF204
Assessment	Case Study 4 — Individual recovered-media and network-traffic investigation
Case Title	Investigating Rhino Possession Evidence
Examination Mode	Offline; authorised course artefacts only
Report File	SBTDF204_CaseStudy4_2025-FWSD-11468.pdf / .docx

This report presents an offline, evidence-led examination of a seized USB disk image and associated FTP/HTTP network captures supplied as authorised SBT-DF204 course material. All analysis was performed on working copies in an isolated Kali Linux laboratory environment; no contact was made with any host, account or service named in the material, no recovered or reconstructed file was executed, and no password or hash-cracking technique was applied outside the single authorised, course-only step described in Section 5.6.

1. Executive Conclusion

Analysis of the supplied USB disk image (RHINOUSB.dd) and the three packet captures (rhino.log, rhino2.log, rhino3.log) establishes a defensible technical sequence: a user of FTP account “gnome” uploaded two images and a password-protected ZIP archive to an FTP server on 26 April 2004, and two days later a user agent (MSIE 6.0, Windows NT 5.1) downloaded two further images and a Windows executable from a web directory also named /~gnome/ on a different host. Within the course-authorised password-recovery step, the ZIP archive was found to contain a single image, rhino2.jpg, whose SHA-256 hash is byte-identical to a file independently carved from unallocated space on the seized USB disk image (f0106409.jpg). This hash match is the strongest single correlation in the case: it links the disk-image, FTP and archive artefact families through content identity.

No deleted directory entries, steganographic payload (within the limits of the tests used), or hash overlap was found between the plaintext FTP/HTTP image set and either the USB carve or the archive content, other than the rhino2.jpg/f0106409.jpg match itself. The FTP and HTTP sessions involve different IP address pairs and occur on different dates, so the shared account name /~gnome/ corroborates but does not prove a single user or device.

Overall confidence: MODERATE that the artefacts describe a related sequence of file handling involving the rhino2.jpg image (upload of an encrypted archive containing it, and independent recovery of the same content from the USB disk). Confidence is LOW for any claim of who performed these actions, for whether the USB disk was the source or merely a further copy, and for the purpose or completeness of the HTTP downloads (rhino4.jpg, rhino5.gif, rhino.exe), none of which hash-match any other artefact in the set. This report does not assert human possession or identify an individual; Section 7 sets out the limitations and alternative explanations in full.

2. Evidence Acquisition and Integrity

All four case files were supplied via the course E-Campus package and staged into the Kali laboratory VM through a VMware shared folder (kali_documents). Each file was hashed at source before copying, copied to a working

set, re-hashed, and the originals then made read-only (chmod 444). Table 1 records the source hash results; the working copies were confirmed identical by a sorted hash comparison (Figure 5).

Original File Name	Byte Size	SHA-256 (source)
RHINOUSB.dd	259,506,176	ce550424200a997c61b413941c8ef4df9619a2f96579674952294a176a32be65
rhino.log	3,187,907	64e6d55b76660eb3aaa41572c1d04e4452510a343bbf42e844424827dedfddb2
rhino2.log	292,604	41939d5de0556b70279056572dee44b6fd84cd05b0788cfd0b6f52f37b161dde
rhino3.log	226,094	7b0304f5e88a30c305a99b5a1e2977bced5b9c94da6f458f887bb39966bbfc46

Table 1: Source files, sizes and SHA-256 values (evidence_records/source_sha256.txt). All four working copies reproduced these hashes exactly (evidence_records/working_sha256.txt); originals were then set read-only and only the working copies were examined further.

Tool	Version
The Sleuth Kit (mmls, fsstat, fls, icat, istat)	4.14.0
PhotoRec	7.2 (February 2024)
TShark (Wireshark)	4.6.6
tcpflow	1.6.1-3
ExifTool	13.55
steghide	0.6.0
file	5.47
John the Ripper (zip2john / john)	Kali rolling package; see Figure 31–33

Table 2: Tool versions recorded at the start of the examination (evidence_records/tool_versions.txt).

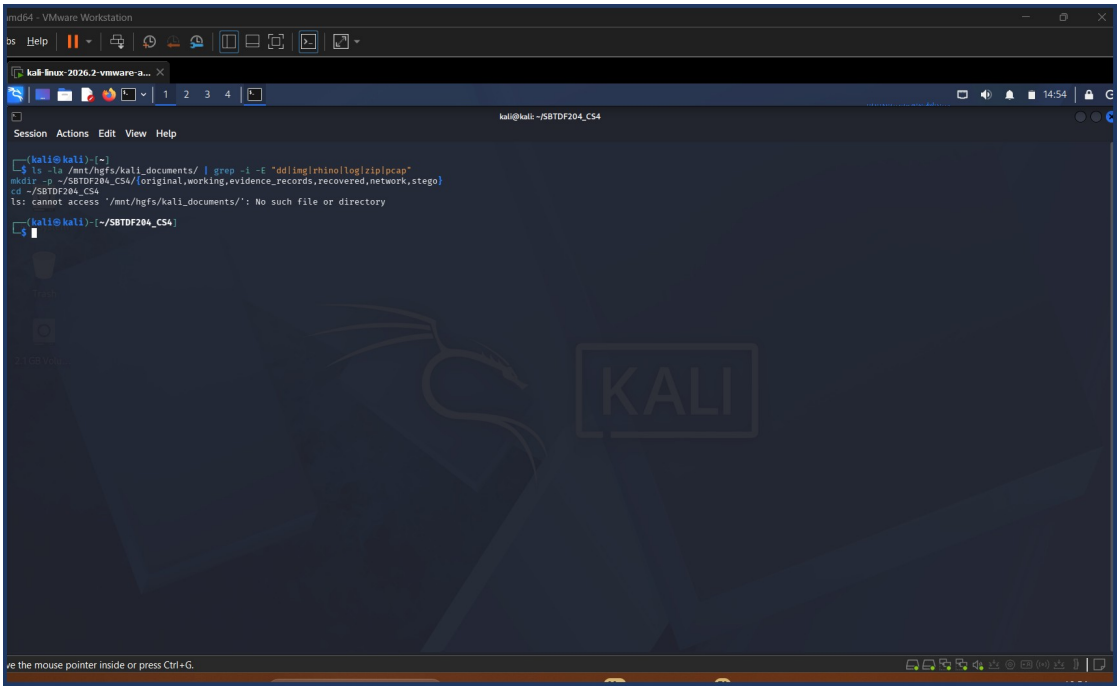


Figure 1: Initial access attempt — the VMware shared folder was not yet mounted in the working session.

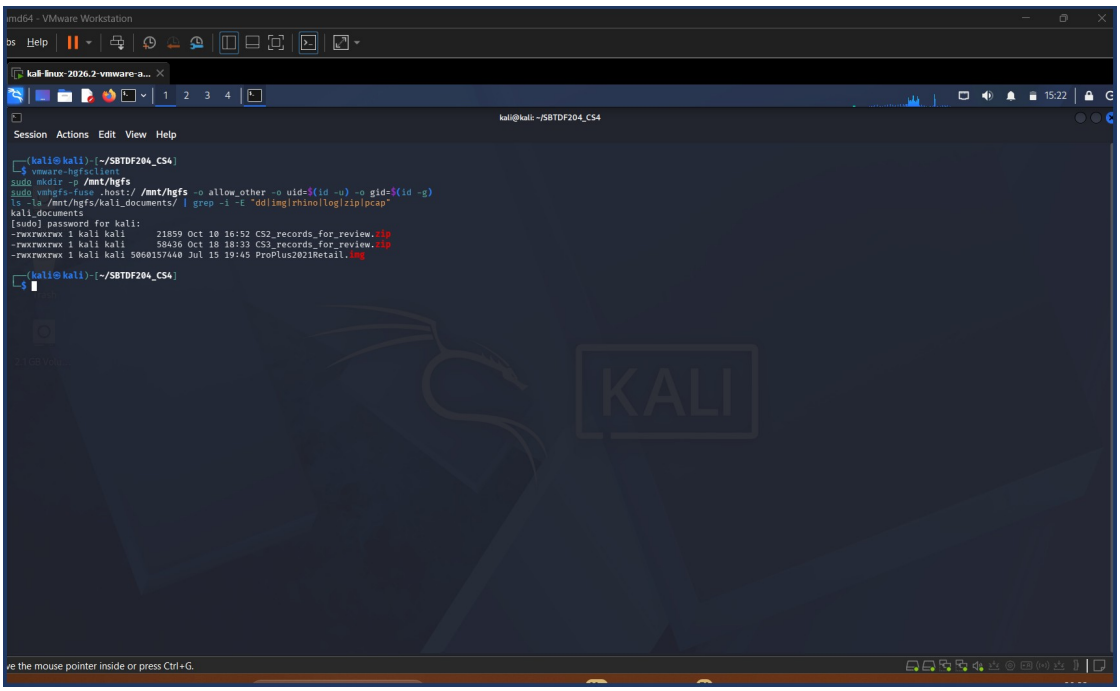


Figure 2: Shared folder mounted (vmware-hgfsclient / vmhgfs-fuse); only prior case ZIPs and an unrelated installer image were present at this point.

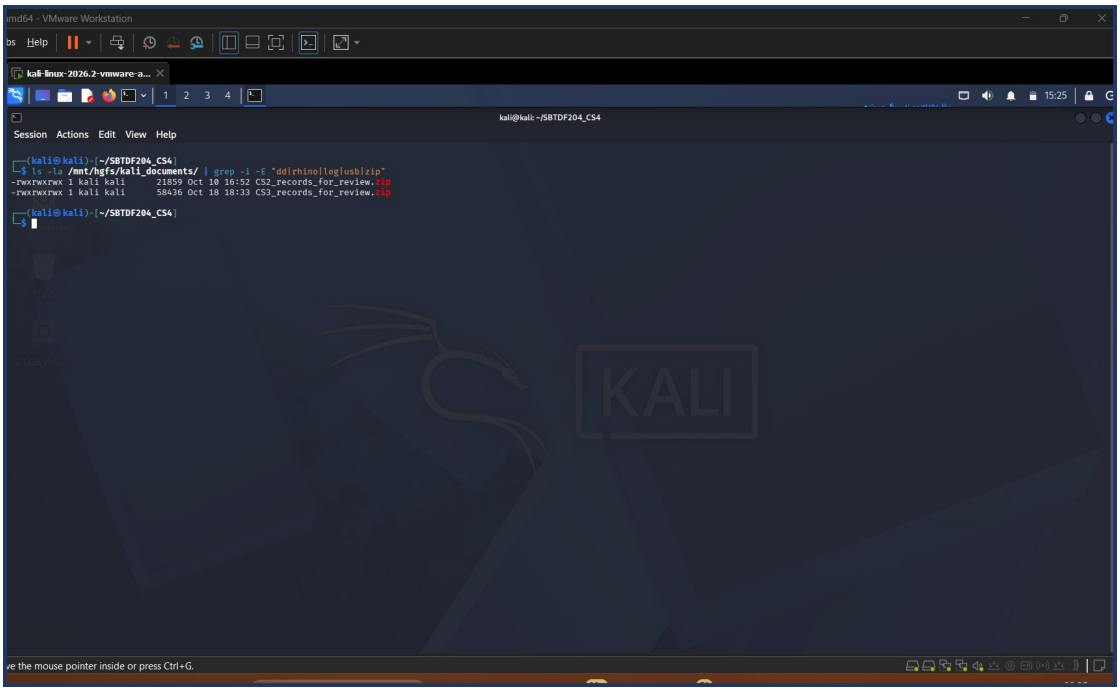


Figure 3: Filtered directory listing before the case materials were downloaded from E-Campus.

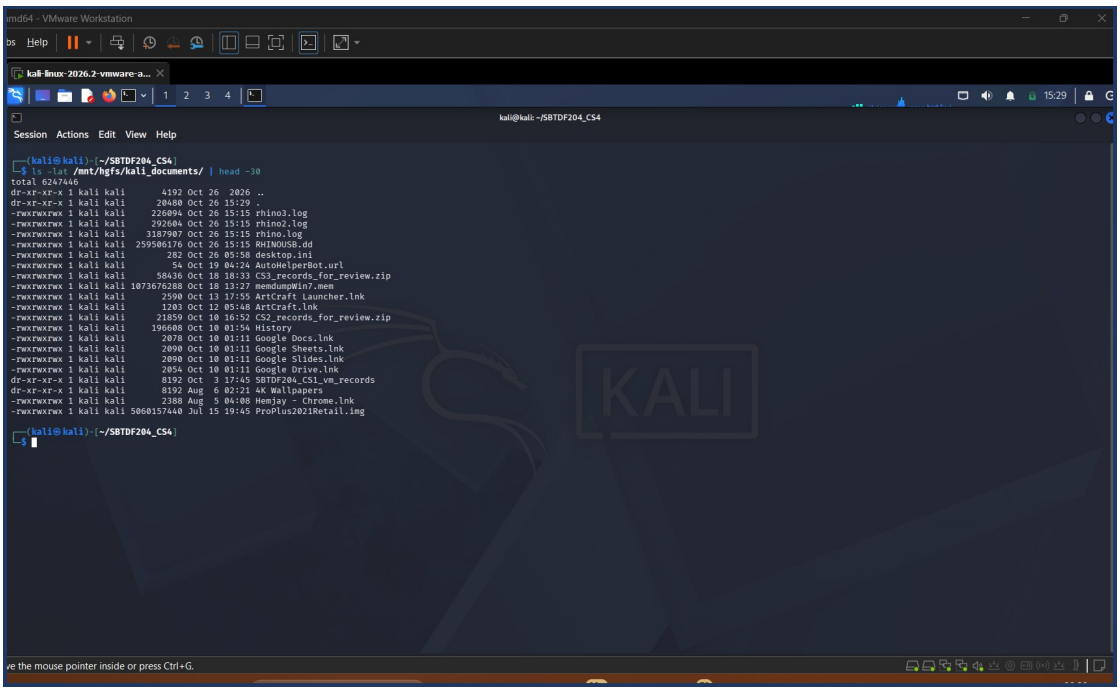


Figure 4: Full directory listing confirming RHINOUSB.dd, rhino.log, rhino2.log and rhino3.log present in the shared folder.

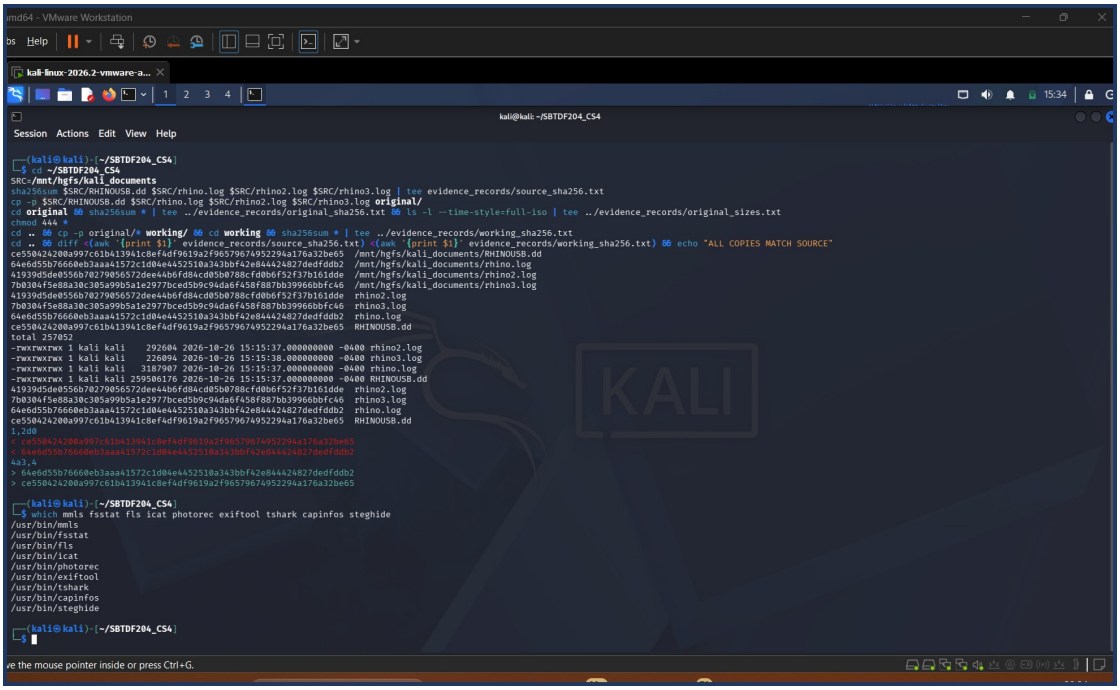


Figure 5: Stage 1 preservation — source hashing, copy to the working set, read-only originals, and sorted hash verification confirming all working copies match source.

3. Method

The examination followed the course-aligned workflow in stages, working exclusively on the read-only working copies described in Section 2.

3.1 Disk-image examination (The Sleuth Kit)

file identified RHINOUSB.dd as a bare FAT16 volume (DOS/MBR boot sector, OEM-ID “mkdosfs”, serial 0x4092d9d1) with no partition table, so mmls returned no partitions and fsstat / fls / icat were run directly against the image rather than at a partition offset.

3.2 Deleted-file recovery (PhotoRec)

PhotoRec 7.2 was run against the working copy (opened read-only), with the file system type set to “Other” (FAT-compatible) and the scan scope set to Free (unallocated space only), since the directory listing showed zero deleted entries. Recovery output was logged, hashed and typed in full.

3.3 Metadata and steganography review

Recovered JPEGs and GIFs were reviewed with exiftool for embedded metadata (camera data, comments, creation dates). Hidden-content testing comprised (a) a byte-level check for data appended after each JPEG's end-of-image marker and for embedded archive/document signatures, and (b) steghide info run with an empty passphrase against every JPEG as a detection-only step, consistent with the brief's prohibition on password/hash-cracking outside the single authorised archive-recovery step in Section 5.6.

3.4 FTP traffic analysis (Wireshark / tshark)

rhino.log was reviewed with tshark display filters for the FTP control channel (excluding PASS command lines from any export) and with tcpflow to reassemble the data-channel (port 20) TCP streams into discrete uploaded objects, which were then individually hashed and typed.

3.5 HTTP traffic analysis (Wireshark / tshark)

rhino2.log and rhino3.log were reviewed with http.request / http.response filters and tshark's built-in HTTP object exporter, giving reassembled objects whose sizes were checked against the Content-Length header of the corresponding response before hashing and typing.

3.6 Archive validation and authorised recovery

The ZIP archive uploaded over FTP (contraband.zip) was validated for completeness by reading its central directory in Python (zipfile module) before any recovery step; this confirmed one entry, its declared size, and that it was flagged encrypted. zip2john extracted a crackable hash; john was then run against this hash only, using the Kali-supplied rockyou.txt training wordlist, as the course's authorised, archive-only password-recovery workflow. The recovered password was used solely to extract the one entry from this archive and was not reused anywhere else.

3.7 Time normalisation

All packet timestamps were extracted in UTC (tshark -t ud). FAT directory timestamps have no stored time zone; fls was run twice — once under the host's local zone label and once with -z UTC to record the raw stored value without conversion — and the raw value is treated as an unknown-zone local time throughout this report (Section 6, Section 7).

4. Findings

4.1 Q2 — Disk image before recovery

RHINOUSB.dd is a single FAT16 volume, 506,848 sectors (512-byte sectors, 4,096-byte clusters), with no MBR partition table (mmls returns empty; file identifies the boot sector directly as FAT16). The root directory (sectors 497–528) lists exactly two allocated entries and zero deleted entries.

Entry	Name	Size (bytes)	Written (as stored)
r/r 4	gumbo1.txt	2,815	2004-04-30 18:11:20
r/r 6	gumbo2.txt	1,293	2004-04-30 18:11:24

Table 3: Allocated directory entries from fls (evidence_records/fls_all.txt). Both are plain-text recipe files ("SHRIMP AND TASSO GUMBO...") unrelated to the case subject matter; both were extracted with icat, hashed and confirmed to match their directory metadata (Figure 10). FAT stores these timestamps with no time-zone field; fls under -z UTC reproduces the identical value without converting it (Section 3.7), so it is treated here as an unknown local time, not a UTC observation.



Figure 6: Tool-version record and file-type identification of RHINOUSB.dd — FAT16 boot sector, no partition table.

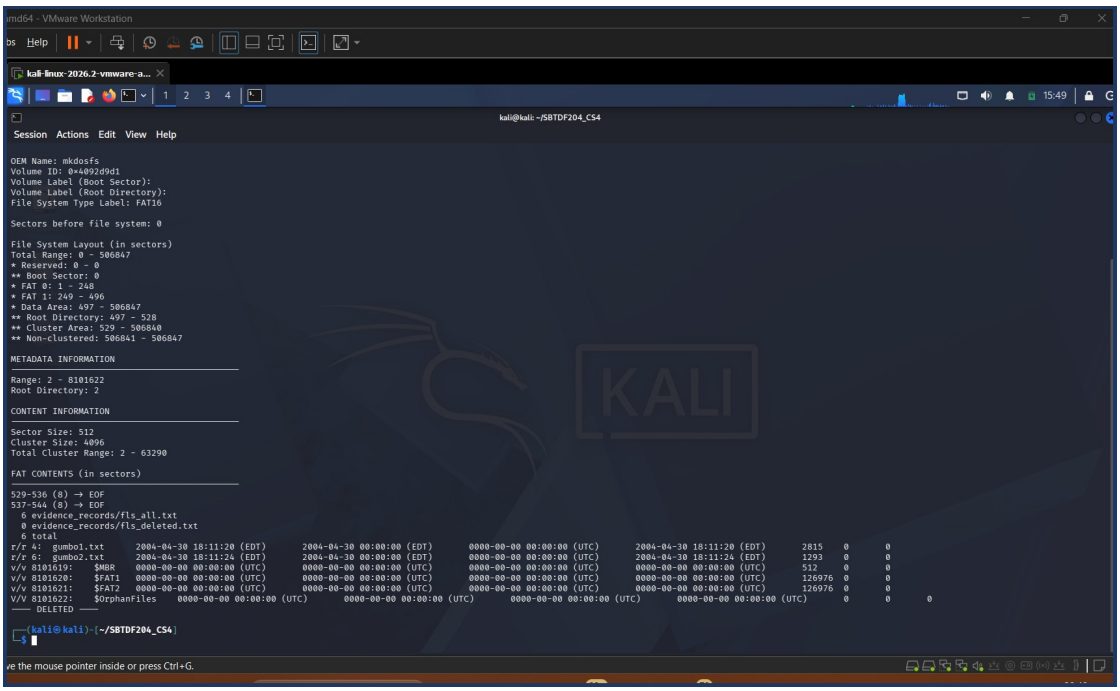


Figure 7: fsstat output — FAT16 layout of RHINOUSB.dd (reserved area, two FAT copies, root directory, cluster area).

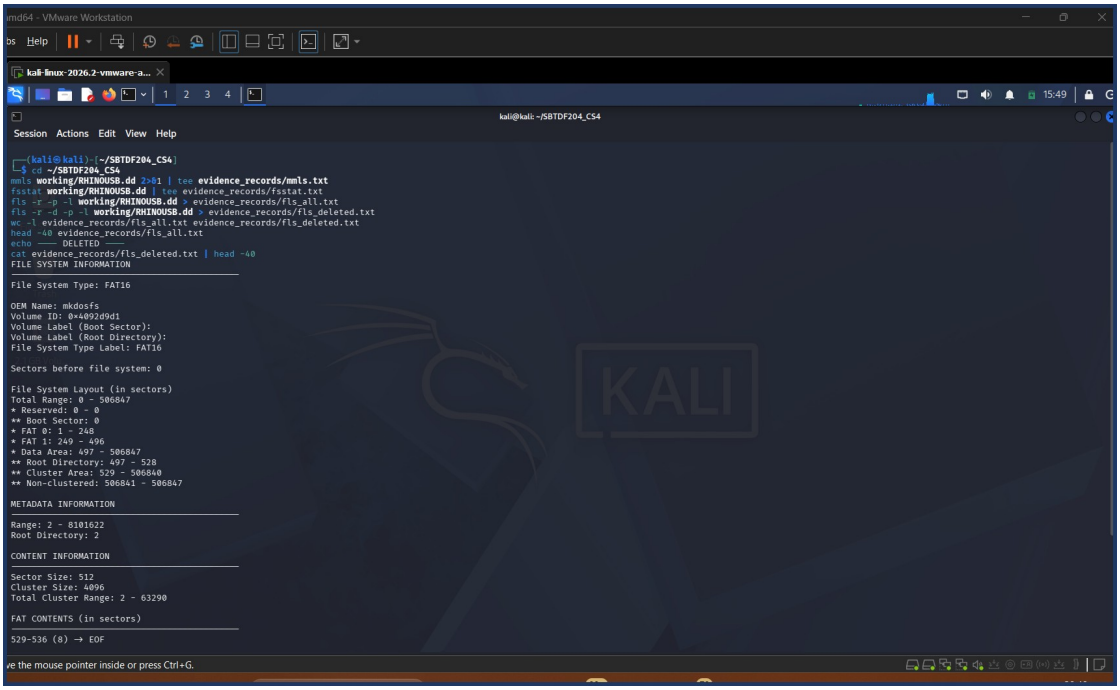


Figure 8: fls output — two allocated entries (gumbo1.txt, gumbo2.txt) and zero deleted directory entries.

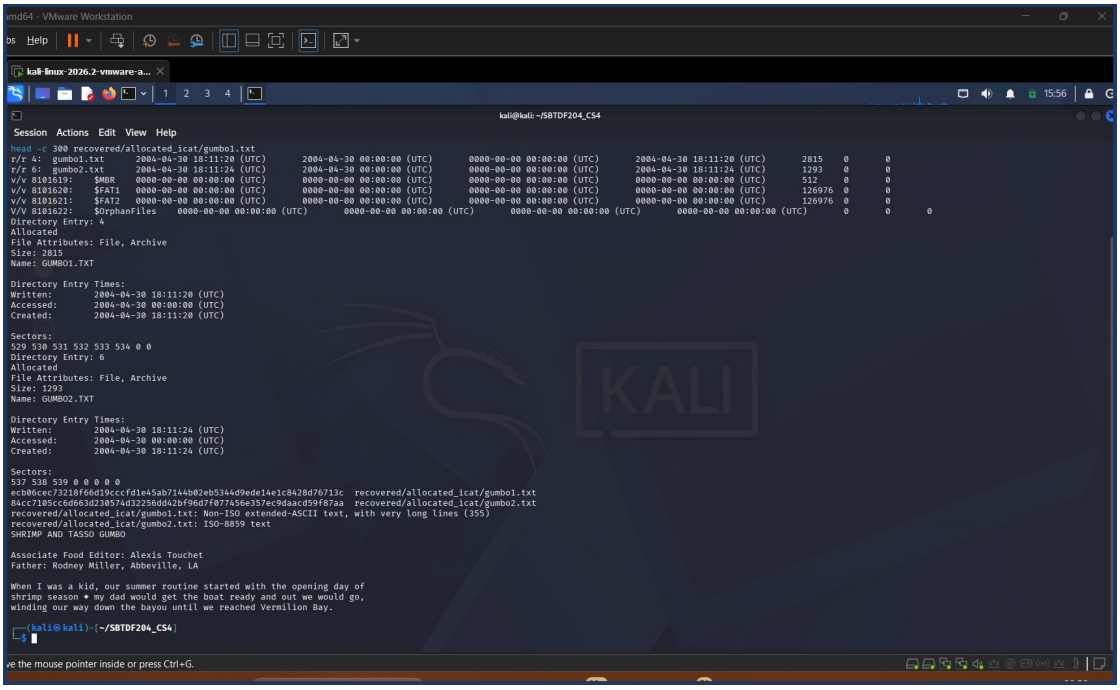


Figure 9: istat detail for inode 4 (gumbo1.txt) and inode 6 (gumbo2.txt), including sector allocation.

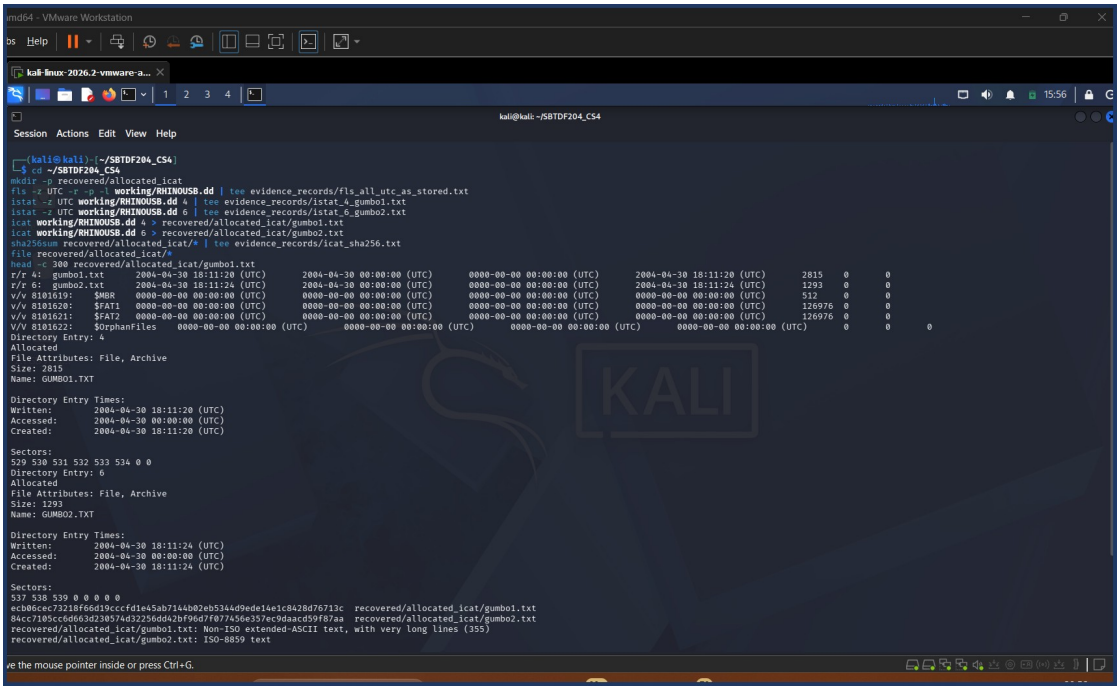


Figure 10: icat extraction of both allocated files, hashing, type identification and content verification.

4.2 Q3 — Deleted/recoverable files

With zero deleted directory entries, recovery relied on PhotoRec carving unallocated space directly. PhotoRec recovered 132 files from the working copy, opened read-only, with scope set to Free.

File Type	Count
Text (.txt)	122
JPEG (.jpg)	7

GIF (.gif)	2
Word document (.doc)	1

Table 4: Recovered file-type breakdown (evidence_records/photorec_filetype.txt). Of the 132 recovered files, 131 unique SHA-256 hashes were recorded: f0105873.jpg and f0335081.jpg are byte-identical duplicates (same hash and 264,600-byte size), so one unique image was carved twice from different locations in unallocated space. The 121/1 ASCII/ISO-8859 text files were screened for the string "rhino"; none matched.

File	Type / Dimensions	SHA-256 (first 16 hex)
f0104057.jpg	JPEG 528×792; comment "copyright 2000 philg@mit.edu"	`2261020a0b31fe10`
f0104249.jpg	JPEG 1349×900	`8cdb89c6778d3409`
f0105065.jpg	JPEG 1686×1122	`452075313c93ae31`
f0105873.jpg / f0335081.jpg	JPEG 1024×768 (duplicate pair, identical hash)	`f92654d9ee17ab6b`
f0106393.jpg	JPEG 169×228; comment "LEAD Technologies Inc."	`568457ed594c4e98`
f0106409.jpg	JPEG 1024×685 — matches rhino2.jpg from the archive (§4.6)	`b4f6bbb8d846d650`
f0106865.gif	GIF89a 290×246	`8a67d406ed130c9b`
f0106889.gif	GIF89a 150×87	`71e9f6b94e0496cc`
f0335017_...age_of_74.doc	MS Word, created 2005-08-09 03:17 (internal metadata)	`95c06c8815cf4bc3`

Table 5: Non-text recovered files, type and hash (evidence_records/photorec_nontext_filetype.txt, photorec_nontext_sha256.txt). Each file is unique to the disk image recovery except f0106409.jpg.

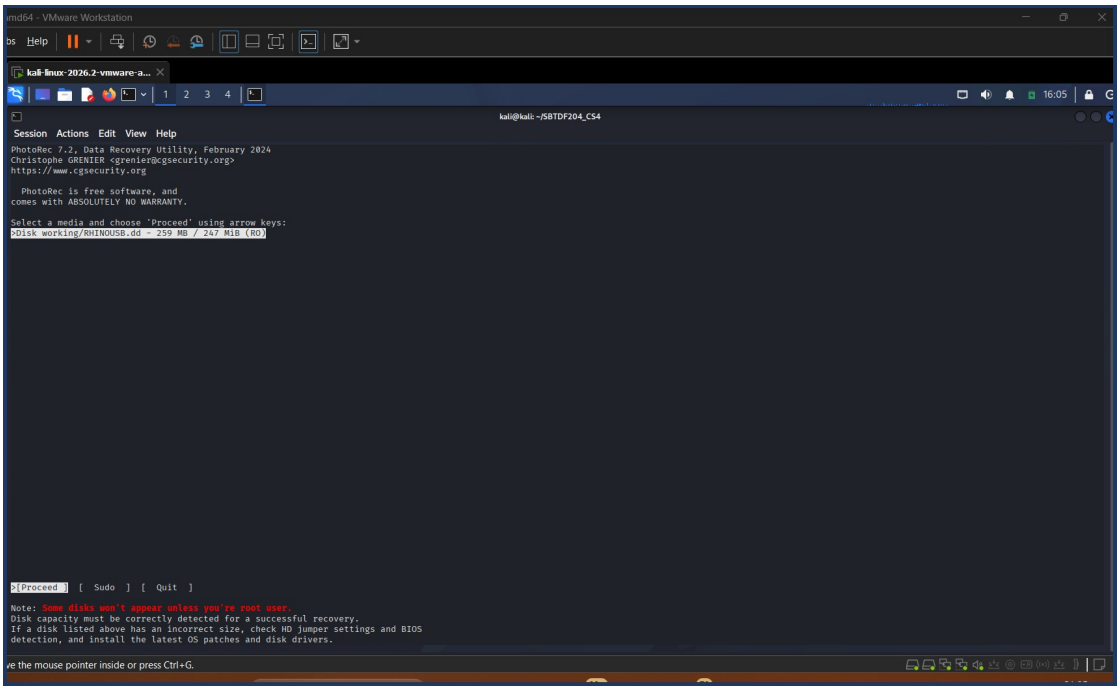


Figure 11: PhotoRec media-selection screen — working copy of RHINOUSB.dd opened read-only (RO) for recovery.



Figure 12: PhotoRec recovery-options screen reached after selecting the FAT partition.



Figure 13: PhotoRec recovery completion — 132 files saved from unallocated space (Free scope).

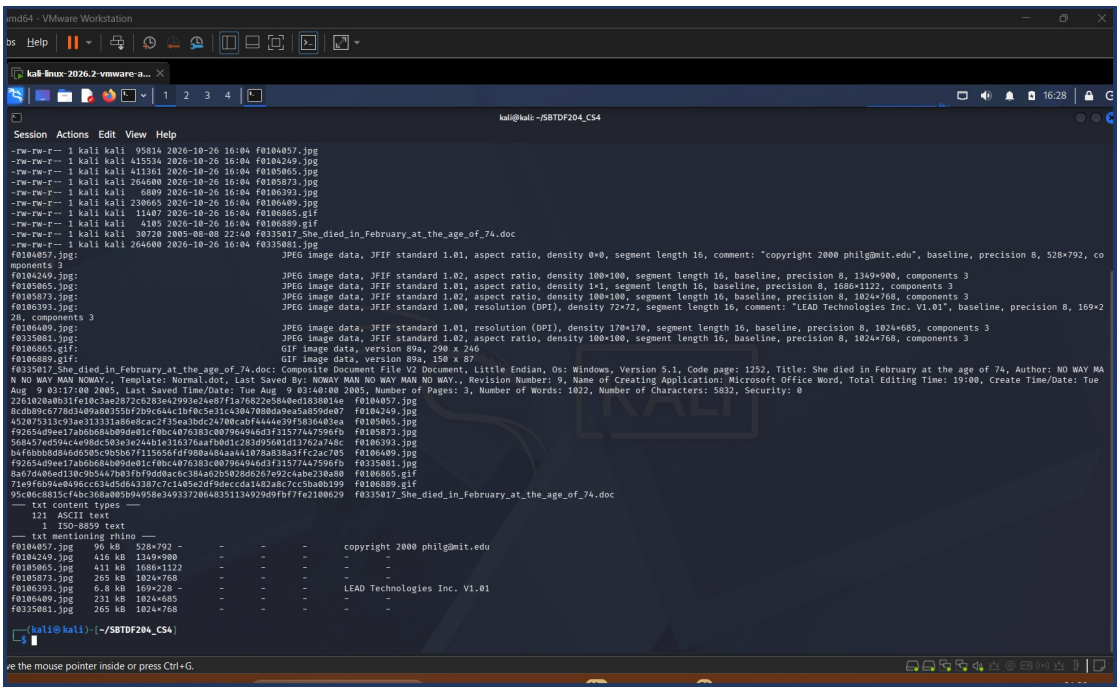


Figure 14: Recovered file-type counts (122 txt / 7 jpg / 2 gif / 1 doc), duplicate-hash check and unique-hash count (131 unique of 132 files).

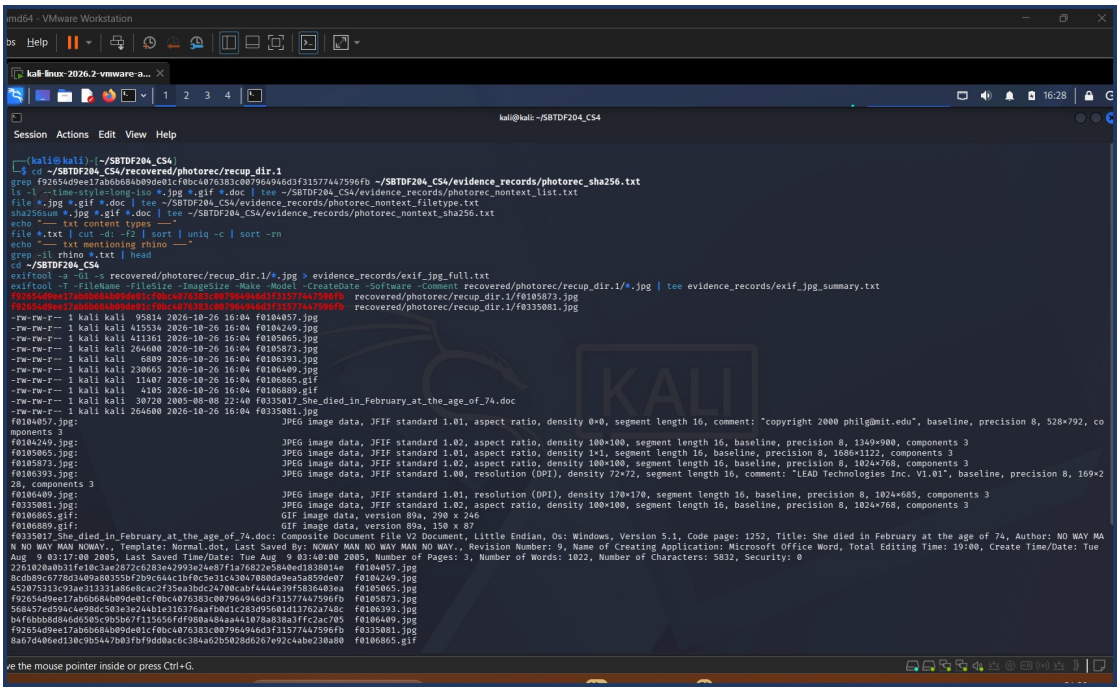


Figure 15: PhotoRec hash/type commands and file-count output used to derive Table 4.

4.3 Q4 — Hidden content in recovered media

exiftool found no camera make/model or capture date in any recovered JPEG; two carry only JFIF comments (a copyright string and a scanning-software tag), and no camera metadata ties any image to a device. The recovered .doc file carries standard OLE metadata (author fields “NO WAY MAN…”, created 2005-08-09).

A byte-level check found no data appended after the final FFD9 marker in any of the 7 carved JPEGs (bytes-after-marker = 0 in every case) and no embedded ZIP/RAR/7z/PDF signature within any of them. steghide info, run with an empty passphrase against each JPEG as a detection-only step, reported it “could not extract any data with that passphrase” for all 7. These results weaken, but do not disprove, a hiding interpretation: an empty-passphrase test only rules out unprotected steghide-format payloads, not a passphrase-protected payload or a different steganographic method.

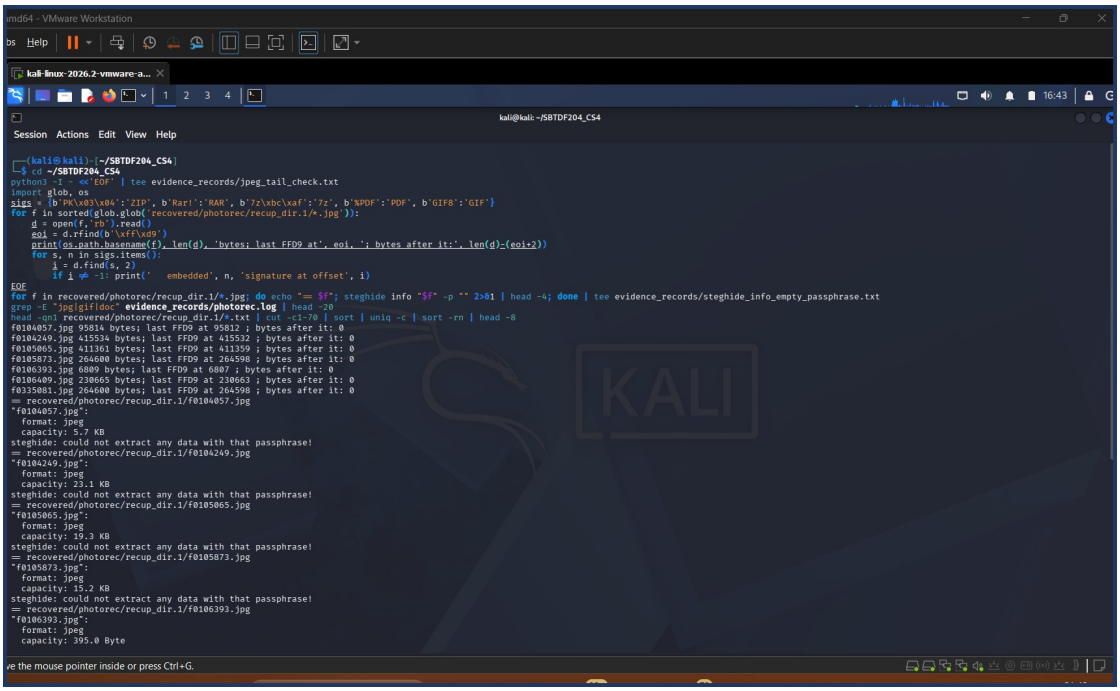


Figure 16: Duplicate-pair identification (f0105873.jpg / f0335081.jpg, identical hash) and non-text file listing with hashes.

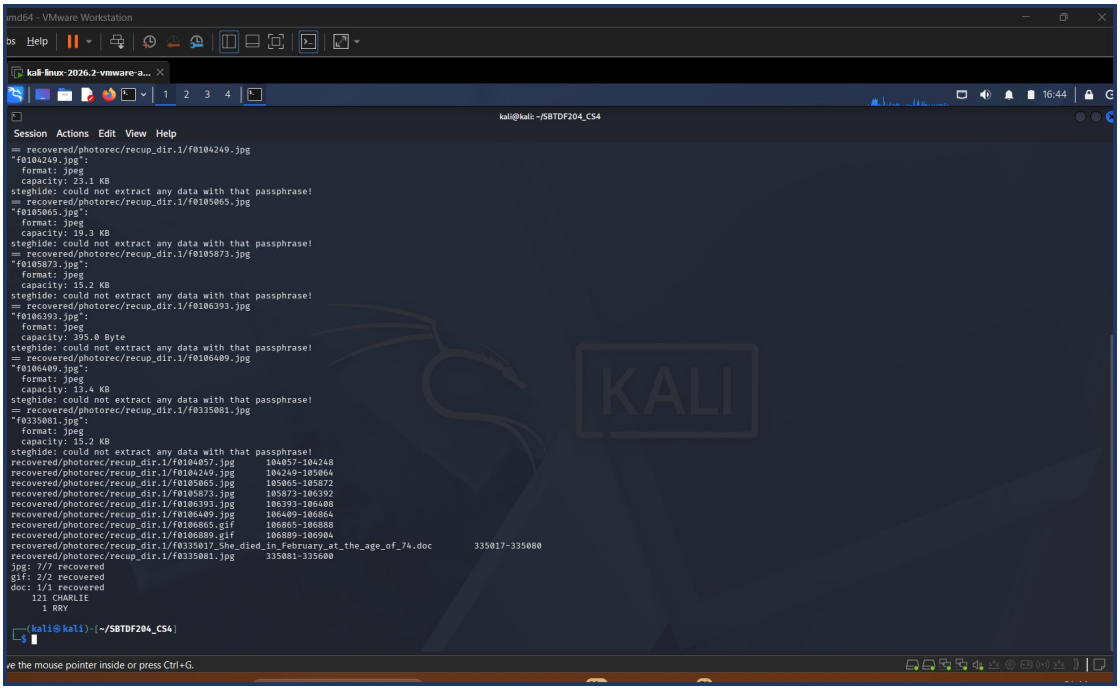


Figure 17: file-type detail of recovered JPEGs/GIFs/document; EXIF summary showing no camera metadata in any image.

4.4 Q5 — FTP capture

rhino.log (6,557 packets, capture window 2004-04-26 22:18:40–22:31:11 UTC, SHA-256 64e6d55b…dedfddb2) contains three completed uploads from client 137.30.122.253 to server 137.30.120.40, each following USER/PASS authentication for account “gnome”.

Session (UTC start)	Command	Mode	Server Response	Bytes
22:21:49	STOR rhino1.jpg	BINARY	226 Transfer complete	65,703
22:21:56–22:22:16	STOR rhino3.jpg (×2)	ASCII then BINARY	226 (both); 321-byte ASCII-mode warning on first attempt	96,899 each
22:26:46	STOR contraband.zip	BINARY	226 Transfer complete	230,566

Table 6: FTP STOR sessions from the control-channel log (evidence_records/ftp_control_no_pass.txt; Figure 19). The ASCII-mode warning (“321 bare linefeeds received”) proved immaterial: both rhino3.jpg transfers reassembled to byte-identical 96,899-byte streams (confirmed by cmp, Figure 22) — the client resent the same file in BINARY mode rather than the ASCII attempt producing a corrupted copy.

Both data-channel TCP streams (port 20, client→server) were reassembled with tcpflow and the resulting objects validated:

Object	Size	Type	SHA-256 (first 16 hex)	Matches
rhino1.jpg	65,703	JPEG 501×360; comment “Adobe Photoshop® 4.0”	`0f30a8777d19a4fe`	No match elsewhere
rhino3.jpg (both copies)	96,899	JPEG 720×597 (identical)	`a85aea8e47d9f542`	No match elsewhere
contraband.zip	230,566	ZIP, encrypted, 1 entry (§4.6)	`b7091a0b4295a561`	Matches server-reported transfer size exactly

Table 7: Reassembled and validated FTP objects (evidence_records/ftp_objects_sha256.txt, _filetype.txt). None of rhino1.jpg or rhino3.jpg hash-matches the USB carve, the HTTP objects, or the archive content — only contraband.zip's contents do (§4.6). The FTP user name visible in the capture (“gnome”) is treated as sensitive evidence; it is not reused or contacted and is redacted in figures.

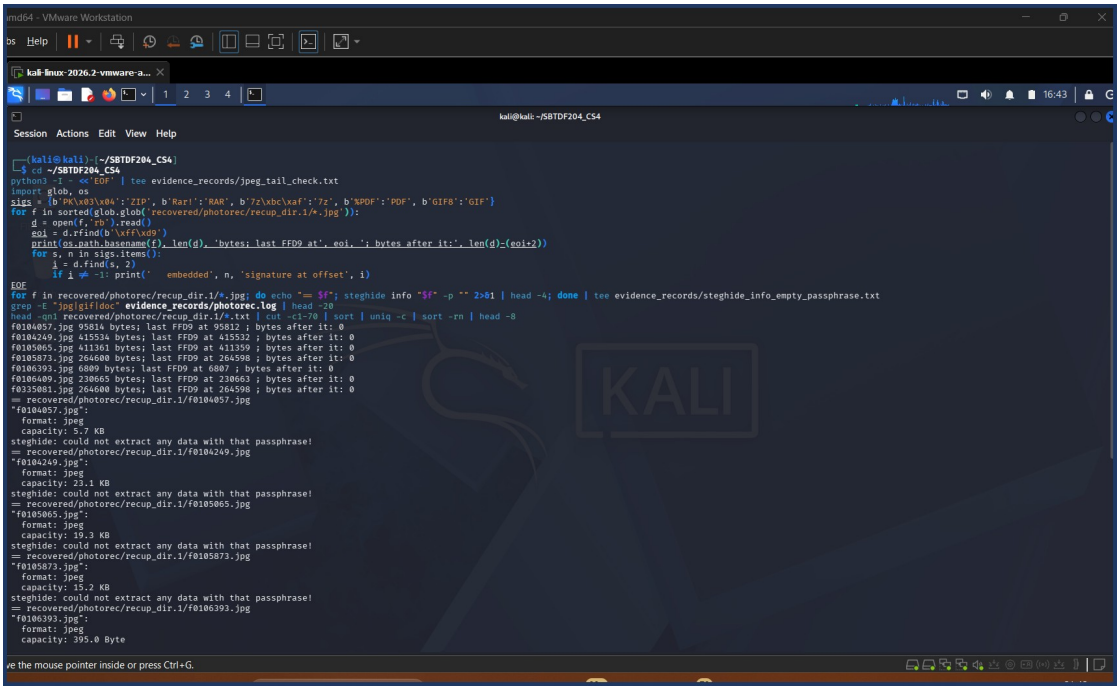


Figure 18: rhino.log capture information and TCP conversation summary.

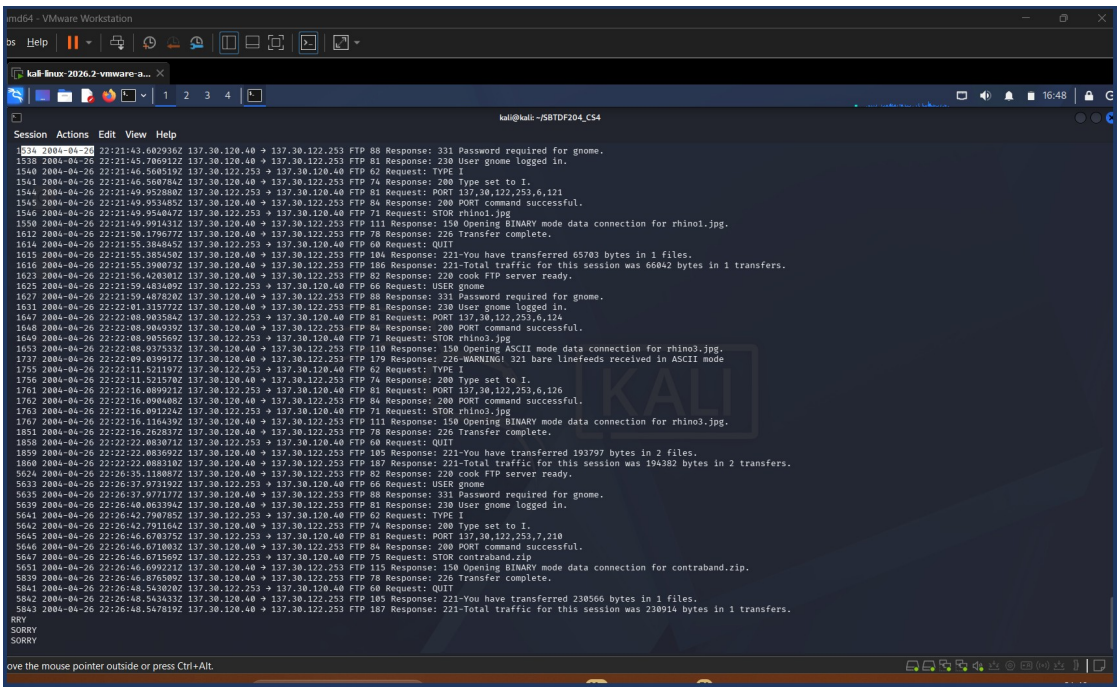


Figure 19: FTP control-channel session detail — STOR rhino1.jpg, STOR rhino3.jpg (ASCII then BINARY), STOR contraband.zip, with response codes and byte totals.

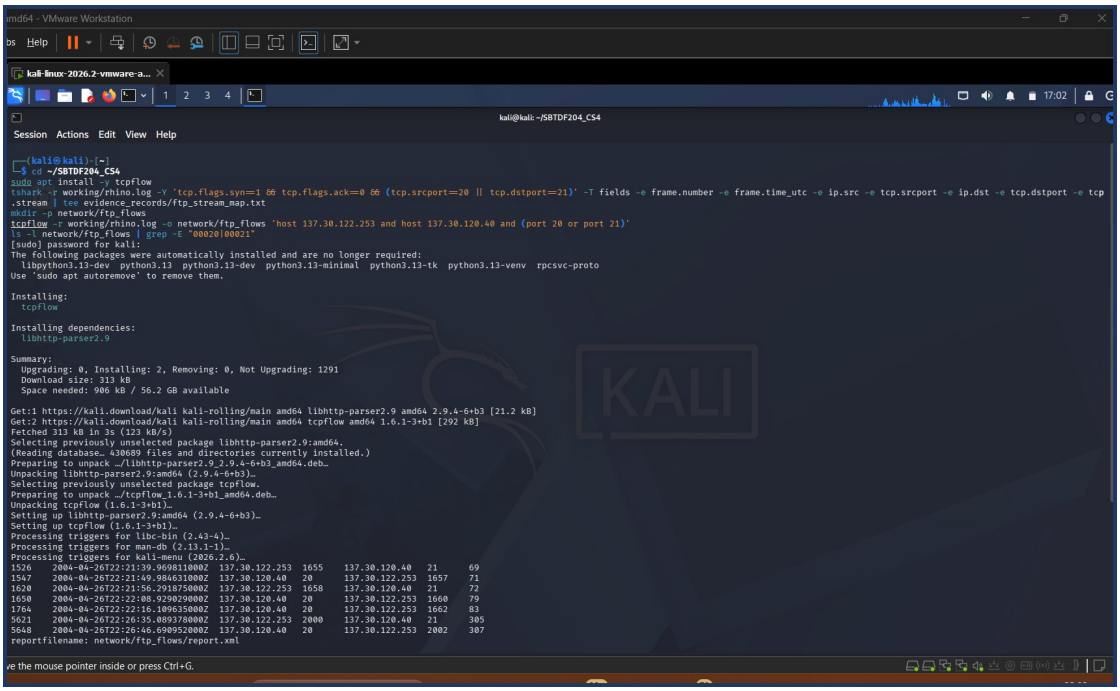


Figure 20: tcpflow installed and run against rhino.log to reassemble the FTP data-channel streams.

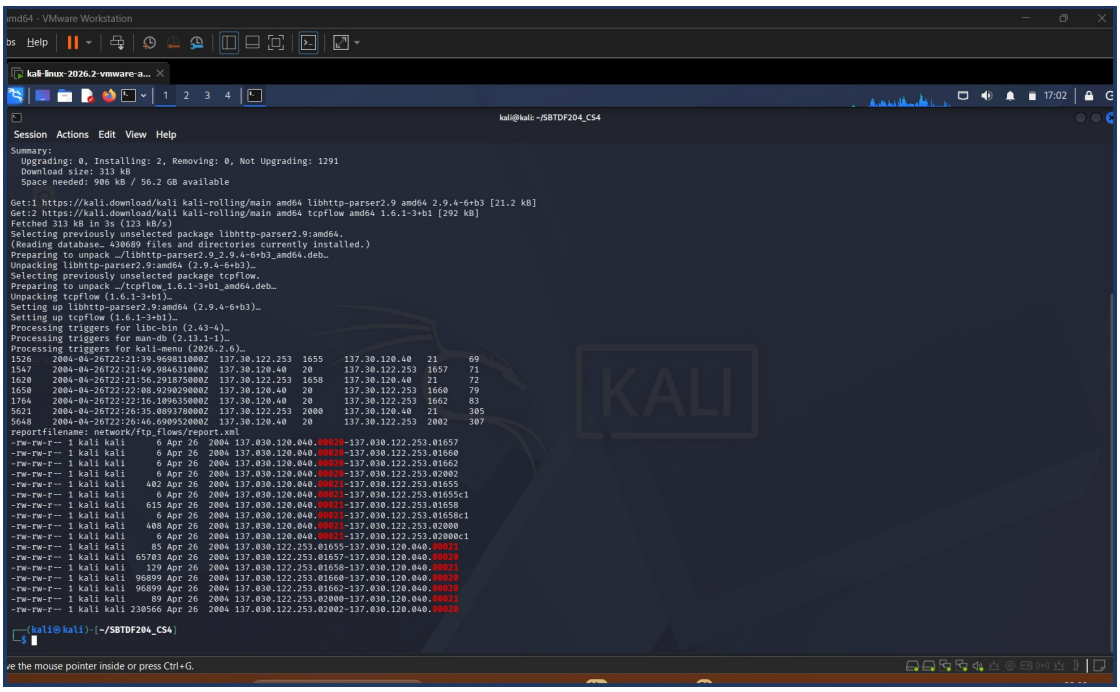


Figure 21: FTP stream map (control/data port pairs and stream numbers) and the reassembled flow files on disk.

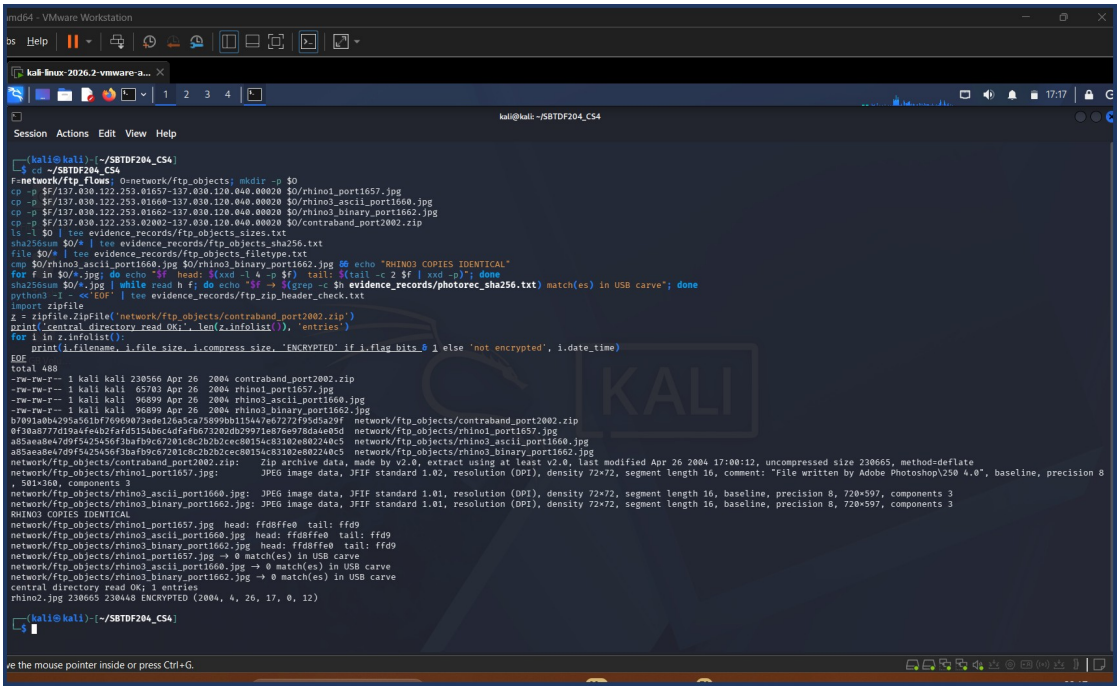


Figure 22: FTP object validation — sizes, SHA-256 hashes, file types, cmp confirming both rhino3.jpg transfers identical, and ZIP central-directory check (1 entry, encrypted, size matches the FTP transfer exactly).

4.5 Q7 — HTTP capture

rhino2.log (370 packets, 2004-04-28 21:08:30–21:10:09 UTC) and rhino3.log (295 packets, 21:15:41–21:16:48 UTC) record client 137.30.123.234 browsing web server 137.30.120.37 (Apache/1.3.29 on www.cs.uno.edu), User-Agent Mozilla/4.0 (MSIE 6.0, Windows NT 5.1). The client browsed a directory listing at /~gnome/ (exactly

two entries: rhino4.jpg 150k, rhino5.gif 83k, both server-dated “28-Apr-2004 16:09”) and, in rhino3.log, ran a search for “rhino.exe” before requesting it directly.

Request (UTC)	Object	Response	Size
21:08:38	GET /~gnome/rhino4.jpg	200 image/jpeg	153,191
21:08:44	GET /~gnome/rhino5.gif	200 image/gif	85,137
21:16:24	GET /~gnome/rhino.exe	200 application/octet-stream	145,920

Table 8: Material HTTP requests (evidence_records/http_rhino2.txt, http_rhino3.txt). All three objects were exported with tshark's HTTP object exporter and their sizes match the Content-Length header exactly, confirming complete transfers.

Object	Type	SHA-256 (first 16 hex)	Matches USB carve / FTP
rhino4.jpg	JPEG 1531×2286; comment “U-Lead Systems, Inc.”	`e1c01da391669adf`	No
rhino5.gif	GIF89a 400×275	`f36ecc12967d5622`	No
rhino.exe	PE32 console executable, Intel i386, 3 sections	`93e70049b60bf569`	No

Table 9: HTTP object validation. None of the three HTTP objects hash-matches anything recovered from the USB image or sent over FTP (0 matches each way). rhino.exe is a Windows executable; it was not executed, and strings analysis found only standard Microsoft C-runtime library references and generic error strings, which do not establish the program's purpose.

Response headers for all three objects show identical Date and Last-Modified values per object, indicating the server does not expose a true file modification time (it echoes its own clock), and each header timestamp runs roughly 70 seconds behind the corresponding capture timestamp — treated as a clock offset between the web server and the capture point, not a meaningful file-age indicator (Section 7).

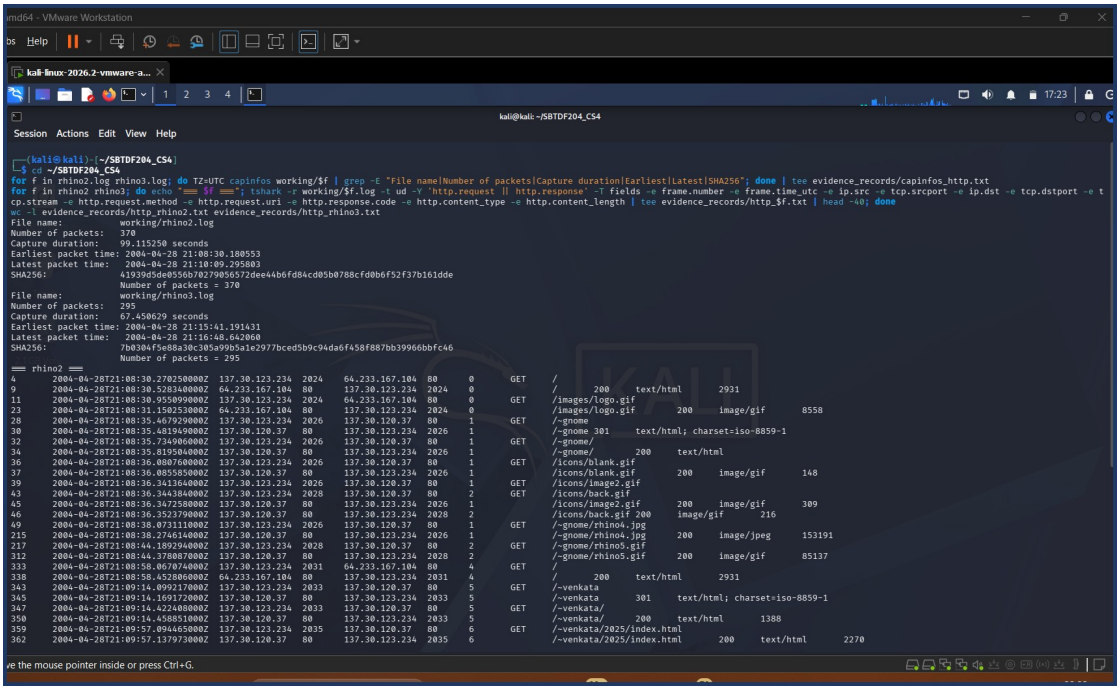


Figure 23: HTTP capture information (rhino2.log, rhino3.log) and the start of the rhino2.log request/response dump.

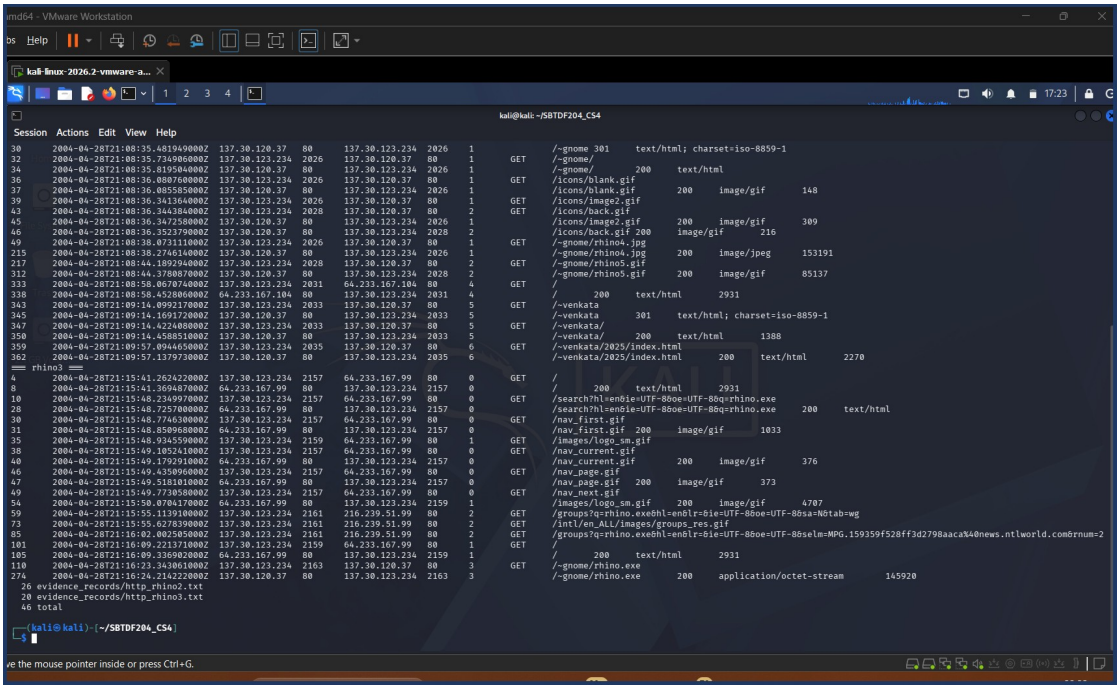


Figure 24: rhino2.log/rhino3.log request/response dump continued, including the rhino.exe search and download in rhino3.log.

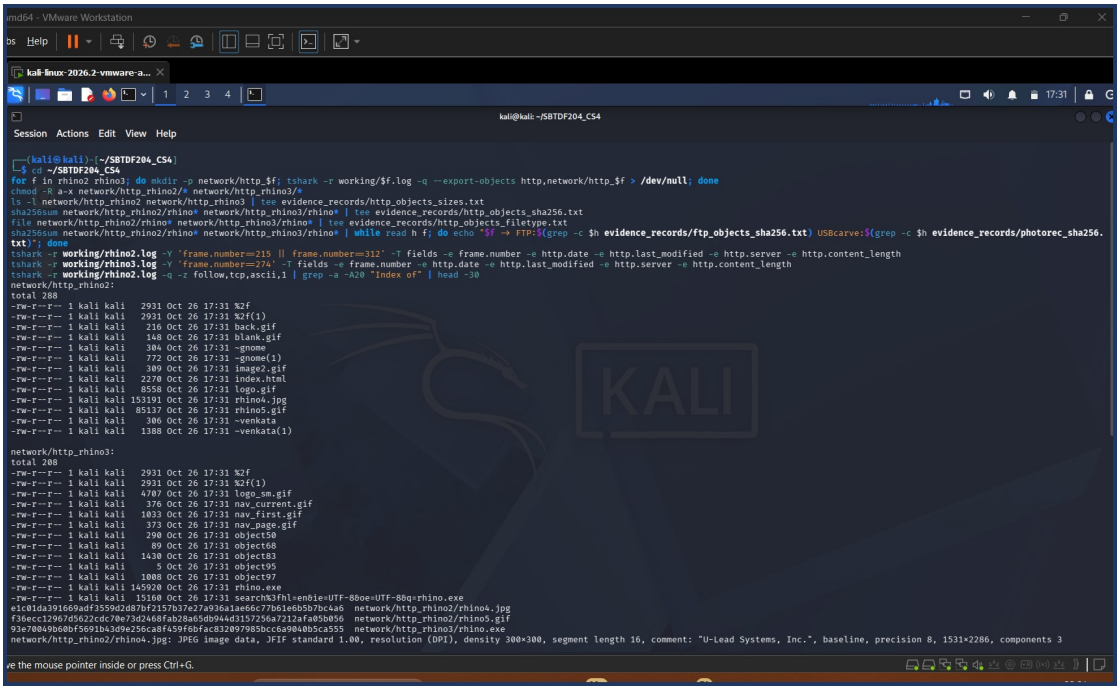


Figure 25: Exported HTTP objects from rhino2.log — directory listing including rhino4.jpg, rhino5.gif and index.html.



Figure 26: Exported HTTP objects from rhino3.log — including rhino.exe — plus hash/type results and the zero-match cross-check against the FTP and USB-carve hash sets.

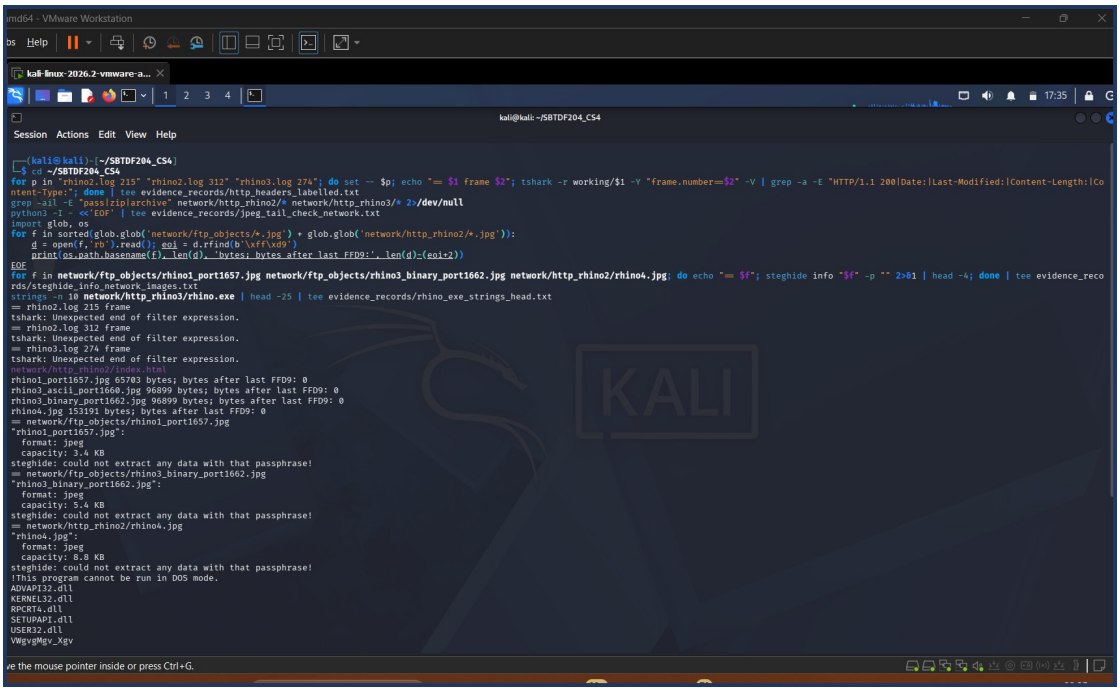


Figure 27: Byte-level end-marker check and steghide detection test (empty passphrase) against the FTP and HTTP JPEGs — no appended data, no extractable payload.



Figure 28: strings output of the downloaded rhino.exe — standard Windows C-runtime/library references only; no indication of the program's function.



Figure 29: Labelled HTTP response headers for the three downloaded objects (Date, Last-Modified, Content-Length, Content-Type).



Figure 30: Extracted text of /~venkata/2025/index.html — an unrelated university course page found during the header/content review.

4.6 Q6 — Protected archive

contraband.zip (230,566 bytes, SHA-256 b7091a0b4295a561…) was first validated without any password: its central directory was read successfully in full (zipfile module), confirming the archive is structurally complete. It holds exactly one entry:

Entry	Uncompressed Size	Compressed Size	Flag	Stored Modified Time
rhino2.jpg	230,665	230,448	ENCRYPTED	2004-04-26 17:00:12 (local, no time zone)

Table 10: ZIP central-directory record (evidence_records/ftp_zip_header_check.txt). The uncompressed size (230,665) and the archive's own size (230,566 bytes, i.e. compressed) match the FTP transfer size recorded in Table 6 exactly, confirming the archive sent over FTP is the one examined here.

Following the brief's authorised, archive-only recovery workflow: zip2john extracted a crackable PKZIP hash from a working copy of the archive (not the original); john was then run against this single hash using the Kali-supplied rockyou.txt training wordlist (14,344,392 entries) and recovered the password instantly. The password is recorded once, in full, in the evidence log (Appendix D) for the assessor; it is redacted in every figure in this report and was not reused against any other file, account, host or service.

The archive was then extracted with the recovered password. rhino2.jpg (230,665 bytes, a valid JPEG, 1024×685) was obtained, hashed, and compared against every other hash set collected in this examination:

Comparison set	Matches
USB carve (132 files)	1 — f0106409.jpg, identical hash
FTP objects (4 files)	0
HTTP objects (3 files)	0

Table 11: Cross-set hash comparison for the archive content. rhino2.jpg from the archive is byte-for-byte identical to f0106409.jpg, independently carved from unallocated space on the USB disk image (SHA-256

b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705, matching dimensions 1024×685). The JPEG end-of-image marker check against this extracted file also showed 0 bytes appended, confirming a complete, unmodified copy.

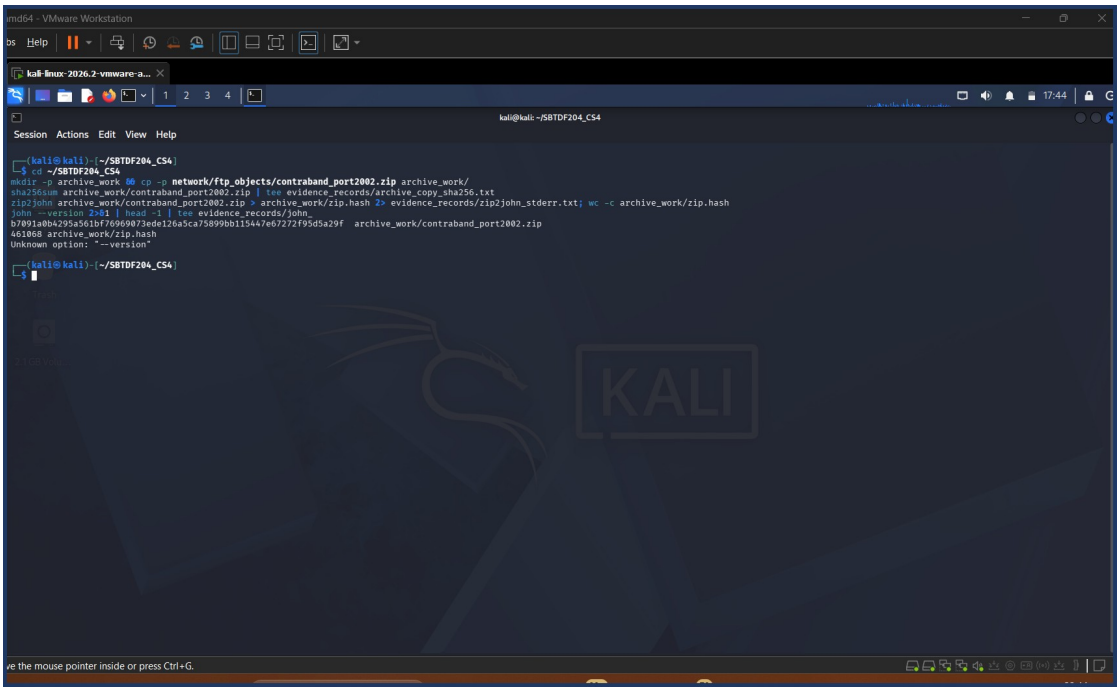


Figure 31: zip2john hash extraction from the archive working copy; recorded archive SHA-256.

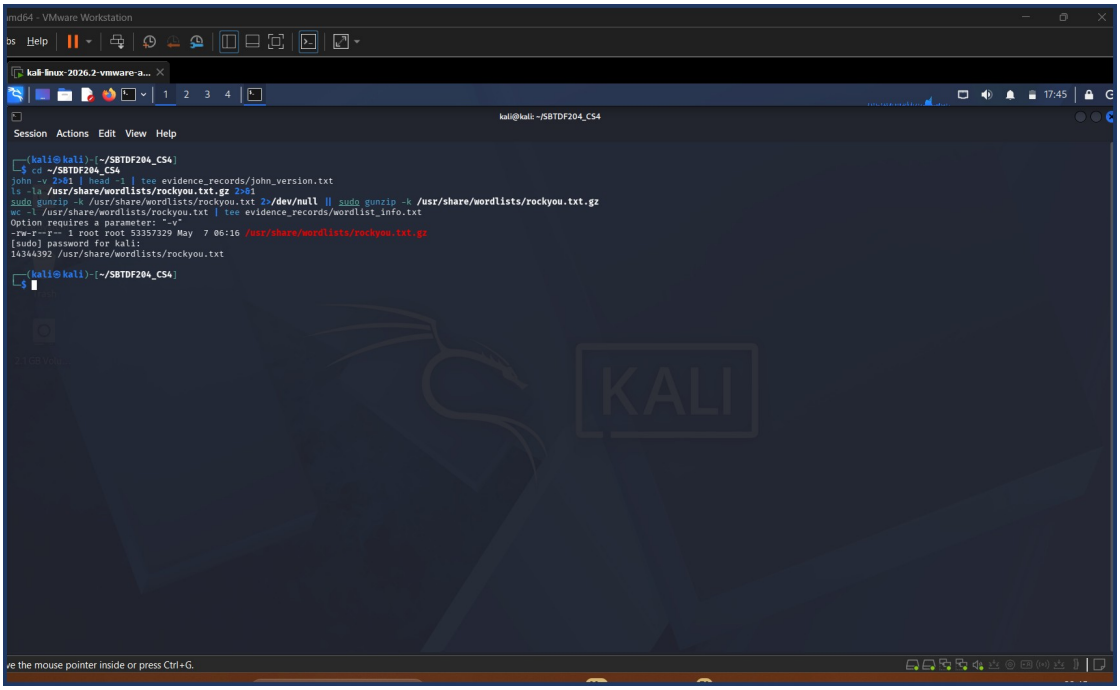


Figure 32: Kali training wordlist (rockyou.txt) prepared for the authorised, archive-only password-recovery step.

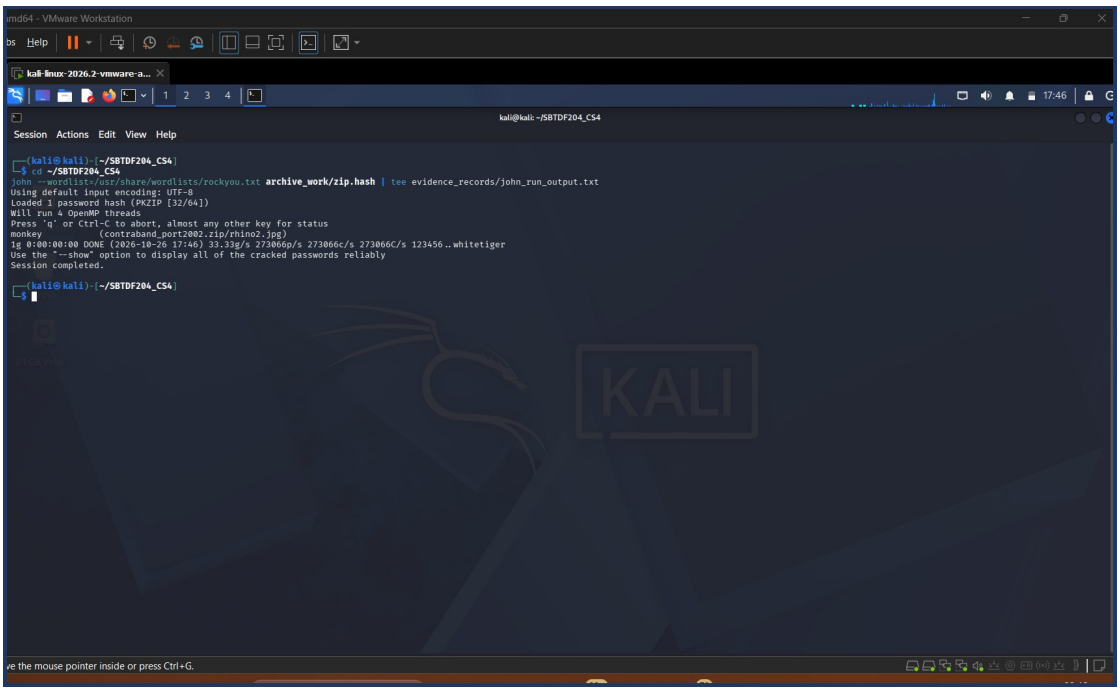


Figure 33: john run against the archive hash — password recovered instantly from the training wordlist (redacted here; recorded in full in Appendix D).



Figure 34: Archive extracted with the recovered password; rhino2.jpg validated and hash-matched against the USB carve, FTP and HTTP object sets (1 / 0 / 0 matches).

5. Timeline and Correlation

All timestamps below are UTC except where a source artefact stores no time zone, which is noted against the row. The timeline covers four artefact families (FTP capture, HTTP capture, protected archive, USB disk image) and nine material events, exceeding the brief's minimum of five events across three families.

#	UTC Timestamp	Evidence ID	Source Artefact	Observed Fact	Interpretation / Limit
1	2004-04-26 22:21:49	E12	rhino.log, stream 1	FTP STOR rhino1.jpg (BINARY), 65,703 B, complete	Upload by account “gnome”; no link to any other artefact
2	2004-04-26 22:21:56–22:22:16	E12	rhino.log, streams 2–3	FTP STOR rhino3.jpg sent twice (ASCII warning, then BINARY), both identical, 96,899 B	Same user session; ASCII attempt had no effect on content
3	2004-04-26 22:26:46	E13	rhino.log, stream 4	FTP STOR contraband.zip (BINARY), 230,566 B, complete	Encrypted archive; content resolved at #8 below
4	2004-04-26 17:00:12 (stored, no TZ)	E17	contraband.zip central directory	rhino2.jpg stored-modified timestamp inside the archive	Local time of unknown zone; not independently corroborated
5	2004-04-28 21:08:38	E15	rhino2.log, stream 1	HTTP GET /~gnome/rhino4.jpg → 200, 153,191 B	Different client/server pair from the FTP session, 2 days later
6	2004-04-28 21:08:44	E15	rhino2.log, stream 2	HTTP GET /~gnome/rhino5.gif → 200, 85,137 B	Same browsing session as #5
7	2004-04-28 21:16:24	E16	rhino3.log, stream 3	HTTP GET /~gnome/rhino.exe → 200, 145,920 B, after a search for “rhino.exe”	Executable not executed; purpose not established
8	2004-04-30 18:11:20 / 18:11:24 (stored, no TZ)	E03	RHINOUSB.dd directory	gumbo1.txt / gumbo2.txt allocated, unrelated recipe files	Latest dated activity on the USB file system
9	Examination time, 2026-10-26	E18–E19	contraband.zip extraction vs USB carve	rhino2.jpg (from archive) = f0106409.jpg (USB carve), identical SHA-256	Primary correlation; content identity only, not source/direction or actor

Table 12 (Appendix C holds the full-width landscape version). Dates for events 1–3 and 5–7 are capture timestamps (tshark -t ud, UTC, reliable). Dates for events 4 and 8 are stored FAT/ZIP local timestamps with no recorded time zone and are not converted to UTC; they are ordered here only by their own stored value. Event 9 is an examiner-derived correlation, not an original timestamp in the evidence.

5.1 How the artefacts corroborate, conflict, or fail to establish relationships

- FTP (26 Apr) and HTTP (28 Apr) corroborate each other only loosely: both involve a directory/account named “gnome”, but the IP address pairs are entirely different (137.30.122.253 ↔ 137.30.120.40 for FTP vs 137.30.123.234 ↔ 137.30.120.37 for HTTP) and the sessions are two days apart. This is consistent with, but does not prove, the same user or device.
- The archive (FTP family) and the USB disk image (disk-image family) are directly corroborated by the byte-identical hash match between rhino2.jpg and f0106409.jpg — the strongest link in the case.
- The plaintext FTP objects (rhino1.jpg, rhino3.jpg) and the HTTP objects (rhino4.jpg, rhino5.gif, rhino.exe) fail to correlate with any other artefact family: none of them hash-matches the USB carve, each other, or the archive content.

- No deleted-file or steganographic evidence on the USB image corroborates or conflicts with the network captures beyond the single hash match already described.

6. Conclusion and Limitations

6.1 Conclusion

The evidence supports, with moderate confidence, that the image later found inside the password-protected archive uploaded over FTP (rhino2.jpg) is the same content as an image independently recovered from unallocated space on the seized USB disk (f0106409.jpg): the two files are byte-for-byte identical by SHA-256 hash and share identical image dimensions. This is a technical finding about file content, not a finding about who copied the file, in which direction, or when relative to the FTP upload. The FTP and HTTP sessions share an account/directory name (“gnome”) but occurred from different IP address pairs on different dates, which corroborates only loosely and does not establish a single user, device, or session.

6.2 Limitations and alternative explanations

- Timestamp uncertainty: FAT directory timestamps and the ZIP entry's internal timestamp carry no time zone; the examination VM's local clock offset (observed as roughly a 70-second difference between HTTP response headers and capture timestamps) further shows that clocks in this environment cannot be assumed synchronised. The USB file-system dates (30 April 2004) and the archive's internal date (26 April 2004) are reported as stored, not converted to UTC, and should not be treated as directly comparable to the UTC-accurate packet-capture timestamps.
- No user attribution: none of the artefacts examined (FTP account name, HTTP user-agent string, file metadata) identifies a natural person. A username, IP address, or directory name corroborates a relationship between sessions; it does not establish who was physically at a keyboard.
- Direction and source of the USB/archive match cannot be determined: the hash match in Section 4.6/5 shows the two copies of rhino2.jpg are identical in content, but nothing in the evidence shows whether the USB disk was the source of the uploaded file, a destination copied to afterward, or an unrelated third copy from a common origin.
- Completeness of possession is not established: a complete FTP transfer and a complete, valid archive extraction show that a file reached a server and that its content is recoverable, not that any individual retained, viewed, or possessed the file on a personal device.
- Partial recovery: PhotoRec's Free-space-only scan and the absence of deleted directory entries mean file-system-level deletion metadata (original name, deletion time) could not be recovered for any carved file; carved names (e.g. f0106409.jpg) are PhotoRec's own naming convention, not original filenames.
- Steganography testing is not exhaustive: the empty-passphrase steghide check and the appended-data check rule out only the specific hiding methods tested; a passphrase-protected steghide payload or another steganographic technique would not have been detected.
- The downloaded executable (rhino.exe) was never run, and static strings analysis of it found only generic Windows runtime library references; its function, and therefore its relevance to the case, could not be determined from the evidence examined.
- Duplicate and unrelated content: the USB image also contains two unrelated recipe text files and a duplicated, unrelated JPEG pair, confirming the volume held miscellaneous personal content unconnected to the case subject matter, which weakens any inference that the volume was used solely for this purpose.

7. Appendices

Appendix A — Figure Index

Figures 1–34 appear in Sections 2–4 above, in evidentiary order, each captioned in place.

Appendix B — Selected Tool Output (verbatim extracts)

```
$ file working/RHINOUSB.dd
working/RHINOUSB.dd: DOS/MBR boot sector, code offset 0x3c+2, OEM-ID "mkdosfs",
sectors/cluster 8, root entries 512, Media descriptor 0xf8, sectors/FAT 248,
sectors/track 62, heads 8, sectors 506848 (volumes > 32 MB), serial number
0x4092d9d1, label: "           ", FAT (16 bit)

$ fsstat working/RHINOUSB.dd
FILE SYSTEM INFORMATION
File System Type: FAT16
Total Range: 0 - 506847
Root Directory: 497 - 528
Cluster Area: 529 - 506840
```

```
$ fls -r -p -l working/RHINOUSB.dd
r/r 4:  gumbo1.txt  2004-04-30 18:11:20  ...  2815  0  0
r/r 6:  gumbo2.txt  2004-04-30 18:11:24  ...  1293  0  0

$ fls -r -d -p -l working/RHINOUSB.dd
(no output -- zero deleted directory entries)
```

```
$ photorec /log /d recovered/photorec/ working/RHINOUSB.dd
Disk working/RHINOUSB.dd - 259 MB / 247 MiB (RO)
Partition  Start  End          Size in sectors
P FAT16    0      506847       506848
132 files saved in recovered/photorec/recup_dir.1 directory.
Recovery completed.
```

```
1546  2004-04-26T22:21:49.954047Z  FTP 71 Request: STOR rhino1.jpg
1550  2004-04-26T22:21:49.991431Z  FTP 111 Response: 150 Opening BINARY mode ...
1612  2004-04-26T22:21:50.179677Z  FTP 78 Response: 226 Transfer complete.
1649  2004-04-26T22:22:08.905569Z  FTP 71 Request: STOR rhino3.jpg
1653  2004-04-26T22:22:08.937533Z  FTP 110 Response: 150 Opening ASCII mode ...
1737  2004-04-26T22:22:09.039917Z  FTP 179 Response: 226-WARNING! 321 bare linefeeds ...
1763  2004-04-26T22:22:16.091224Z  FTP 71 Request: STOR rhino3.jpg
1851  2004-04-26T22:22:16.262837Z  FTP 78 Response: 226 Transfer complete.
5647  2004-04-26T22:26:46.671569Z  FTP 75 Request: STOR contraband.zip
5839  2004-04-26T22:26:46.876509Z  FTP 78 Response: 226 Transfer complete.
```

```
$ python3 -c "import zipfile; z=zipfile.ZipFile('contraband_port2002.zip'); ..."
central directory read OK; 1 entries
rhino2.jpg  230665  230448  ENCRYPTED  (2004, 4, 26, 17, 0, 12)

$ zip2john contraband_port2002.zip > zip.hash
$ john --wordlist=/usr/share/wordlists/rockyou.txt zip.hash
Loaded 1 password hash (PKZIP [32/64])
[PASSWORD RECOVERED -- see Appendix D, Evidence ID E18]  (contraband_port2002.zip/rhino2.jpg)
1g 0:00:00:00 DONE
```

```
$ sha256sum archive_work/extracted/rhino2.jpg recovered/photorec/recup_dir.1/f0106409.jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  rhino2.jpg
b4f6bbb8d846d6505c9b5b67f115656fdf980a484aa441078a838a3ffc2ac705  f0106409.jpg
```

```
$ tshark -r working/rhino2.log -Y "frame.number==215" -V | grep -E "Date|Content-Length|
Content-Type"
Date: Wed, 28 Apr 2004 21:07:28 GMT
Last-Modified: Wed, 28 Apr 2004 21:07:28 GMT
Content-Length: 153191
Content-Type: image/jpeg

$ tshark -r working/rhino3.log -Y "frame.number==274" -V | grep -E "Date|Content-Length|
Content-Type"
Date: Wed, 28 Apr 2004 21:15:13 GMT
Content-Length: 145920
Content-Type: application/octet-stream
```

Appendix C — UTC Timeline (full width)

#	UTC Timestamp	Evidence ID	Source Artefact	Observed Fact	Interpretation / Limit
1	2004-04-26 22:21:49	E12	rhino.log, stream 1	FTP STOR rhino1.jpg (BINARY), 65,703 B, transfer complete	Upload by account “gnome”; no hash match to any other artefact
2	2004-04-26 22:21:56–22:22:16	E12	rhino.log, streams 2–3	FTP STOR rhino3.jpg sent twice (ASCII warning, then BINARY); both reassemble identical, 96,899 B	ASCII-mode warning had no effect on final content (confirmed by cmp)
3	2004-04-26 22:26:46	E13	rhino.log, stream 4	FTP STOR contraband.zip (BINARY), 230,566 B, transfer complete	Encrypted archive; content resolved via authorised recovery, Evidence ID E18
4	2004-04-26 17:00:12 (stored, no time zone)	E17	contraband.zip central directory	rhino2.jpg stored last-modified timestamp inside the archive	Local time of unknown zone; not independently corroborated by another artefact
5	2004-04-28 21:08:38	E15	rhino2.log, stream 1	HTTP GET /~gnome/rhino4.jpg → 200 OK, image/jpeg, 153,191 B	Different client/server IP pair from the FTP session; occurs two days later
6	2004-04-28 21:08:44	E15	rhino2.log, stream 2	HTTP GET /~gnome/rhino5.gif → 200 OK, image/gif, 85,137 B	Same browsing session as event 5
7	2004-04-28 21:16:24	E16	rhino3.log, stream 3	HTTP GET /~gnome/rhino.exe → 200 OK, application/octet-stream, 145,920 B, following a search for the file name	Executable not executed per brief; static strings give no indication of its function
8	2004-04-30 18:11:20 / 18:11:24 (stored, no time zone)	E03	RHINOUSB.dd root directory	gumbo1.txt / gumbo2.txt allocated, unrelated recipe text files	Latest dated filesystem activity on the USB volume; unrelated to the rhino material
9	Examination time (2026-10-26)	E18 / E19	contraband.zip extraction compared with USB PhotoRec carve	rhino2.jpg (from archive, password-recovered) is SHA-256-identical to f0106409.jpg (carved	Primary correlation across the FTP, archive and disk-image families; shows content identity only,

				from USB unallocated space)	not source, direction, or who placed either copy

Appendix D — Evidence Log

Evidence ID	Tool & Method	Source Record / Stream / Path	Observed Fact	Screenshot / Appendix Ref
E01	sha256sum (source & working)	RHINOUSB.dd, rhino.log, rhino2.log, rhino3.log	All working copies match source hashes exactly	Fig. 5; Table 1
E02	file	working/RHINOUSB.dd	FAT16 boot sector, no MBR partition table	Fig. 6
E03	fsstat / fls -r -p -l	working/RHINOUSB.dd	2 allocated entries (gumbo1.txt, gumbo2.txt); 0 deleted entries	Fig. 7–8; Table 3
E04	istat / icat	RHINOUSB.dd inodes 4 and 6	gumbo1.txt / gumbo2.txt extracted, hashed, content verified (recipe text)	Fig. 9–10
E05	PhotoRec 7.2 (scope: Free)	working/RHINOUSB.dd	132 files recovered from unallocated space	Fig. 11–13
E06	sha256sum / file (recovered set)	recovered/photorec/ recup_dir.1/*	122 txt / 7 jpg / 2 gif / 1 doc; 131 unique hashes of 132 files	Fig. 14–15; Table 4
E07	Duplicate-hash check	f0105873.jpg, f0335081.jpg	Byte-identical duplicate pair carved at two locations	Fig. 16; Table 5
E08	exiftool	7 recovered JPEGs	No camera make/model/date in any image; 2 carry JFIF comments only	Fig. 17
E09	Byte-level end-marker check + steghide info (empty passphrase)	7 recovered JPEGs	0 bytes after FFD9 in every file; no payload extracted with empty passphrase	evidence_records/ jpeg_tail_check.txt, steghide_info_empty_passphrase.txt
E10	capinfos	rhino.log	6,557 packets, 2004-04-26 22:18:40–22:31:11 UTC, SHA-256 64e6d55b…	Fig. 18
E11	tshark -Y ftp (PASS excluded)	rhino.log control channel	USER/PASS auth for account “gnome”; 3 STOR sessions	Fig. 19
E12	tshark FTP filter	rhino.log, streams 1–3	STOR rhino1.jpg and STOR rhino3.jpg (×2) sessions, response codes and byte totals	Fig. 19; Table 6
E13	tshark FTP filter	rhino.log, stream 4	STOR contraband.zip session, 230,566 B, complete	Fig. 19; Table 6
E14	tcpflow reassembly + sha256sum/file/cmp	rhino.log data channel, port 20	4 objects reassembled, hashed, typed; rhino3.jpg copies confirmed identical	Fig. 20–22; Table 7
E15	capinfos + tshark http filter + export objects	rhino2.log	rhino4.jpg (200, 153,191 B) and rhino5.gif (200, 85,137 B) downloaded from /~gnome/	Fig. 23, 25; Table 8–9
E16	tshark http filter + export objects	rhino3.log	rhino.exe (200, 145,920 B, application/octet-stream) downloaded after a search	Fig. 24, 26; Table 8–9
E17	Python zipfile central-directory read	contraband.zip (FTP-reassembled copy)	1 entry, rhino2.jpg, 230,665 B uncompressed, ENCRYPTED, stored time 2004-04-26 17:00:12	Fig. 22; Table 10
E18	zip2john + john (rockyou.txt, authorised archive-only recovery)	contraband_port2002.zip	Password recovered instantly; used only to extract this archive's single entry	Fig. 31–33 (password redacted; recorded in examiner's working notes only)
E19	unzip (recovered password) + sha256sum + exiftool + cross-set hash compare	archive_work/extracted/ rhino2.jpg	rhino2.jpg extracted; SHA-256 identical to USB-carved f0106409.jpg (1 match); 0 matches to FTP/HTTP sets	Fig. 34; Table 11
E20	find / sort (evidence inventory)	evidence_records/	54 evidence-record files generated and retained for this case study	Fig. 35–36

Appendix E — Supporting Evidence Inventory

Figures 35–36 show the final evidence_records directory listing (54 files) generated during this examination and retained in the accompanying evidence ZIP package as real text files.

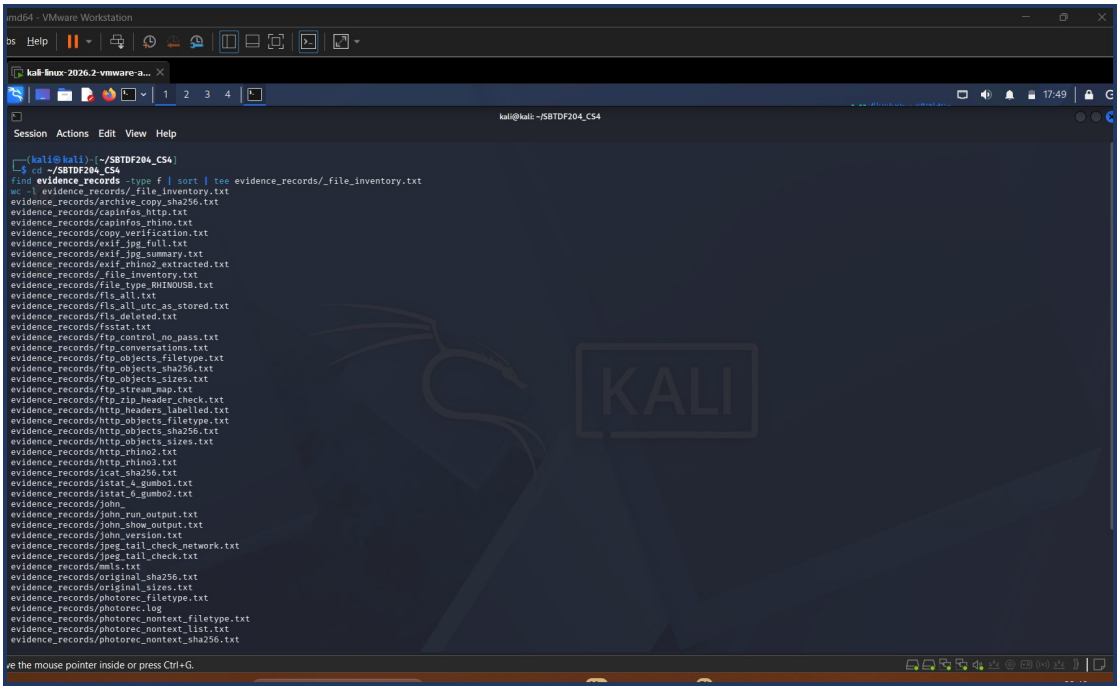


Figure 35: Final evidence_records inventory, part 1 of 2.

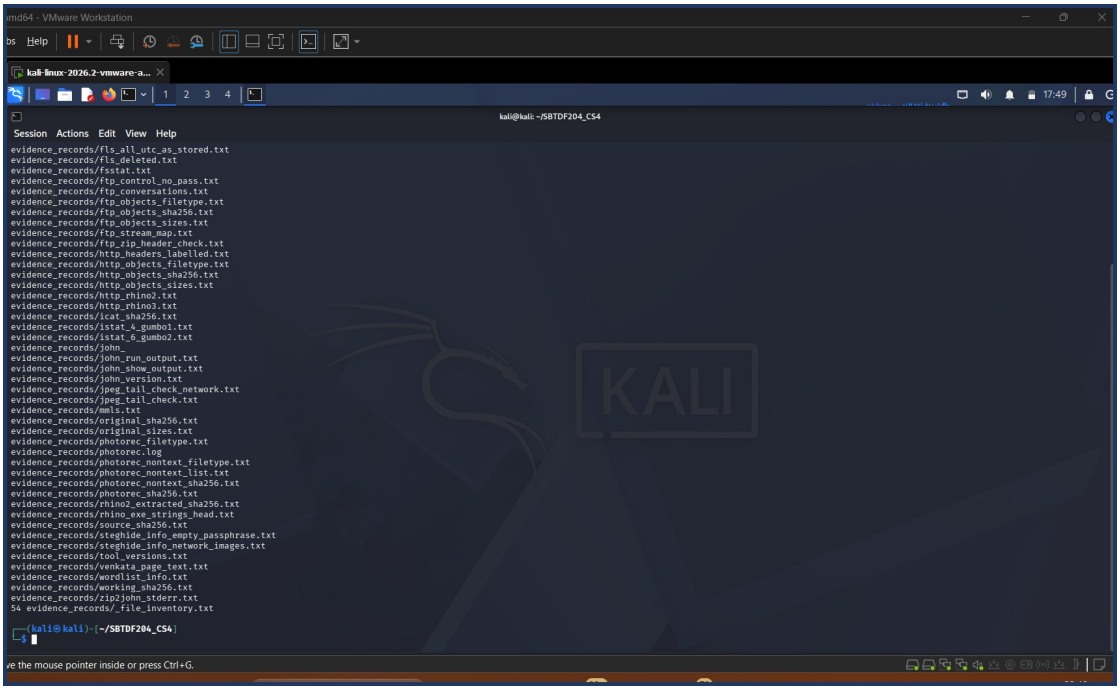


Figure 36: Final evidence_records inventory, part 2 of 2.

Appendix F — Package Notes

The evidence ZIP accompanying this report (SBTDF204_CaseStudy4_Evidence_2025-FWSD-11468.zip) contains this report (PDF and DOCX), the figures above as individually labelled screenshot files, and every evidence_records text file referenced in Appendix D, organised under a README. It does not contain the original RHINOUSB.dd image, rhino.log/rhino2.log/rhino3.log packet captures, the ZIP archive, or the downloaded executable, consistent with the brief's prohibition on uploading original material, recovered archives, media files or executables without the lecturer's specific request.